

Solution Requirements

Date	06 July 2026
Project Name	Credit Card Approval Prediction System
Prepared By	Joyal Kumar, Keerthi, Yasawini, Nasir, Harika

Define Functional and Non-Functional Requirements

A comprehensive list of requirements defining exactly what the Credit Card Approval Prediction system does and how well it performs, based on its current implementation (a Flask web app serving a Random Forest model trained on applicant and credit history data).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	No login is required in the current version - the prediction form is publicly accessible. A future release would add username/password (or bank SSO) login for staff who review flagged applications.	Low
2	Authorization levels	Currently a single implicit role: Applicant (anyone submitting the form). No admin or reviewer dashboard exists yet; a future version would add an Applicant role (submit only) and a Credit Officer role (view predictions, override borderline decisions).	Low
3	External interfaces	No third-party APIs or external databases are used at runtime. The system is self-contained: it reads application_record.csv and credit_record.csv locally and loads the trained model from model.pkl. No payment gateway or bank core-banking integration is currently connected.	Low
4	Transactions processing	Each form submission is processed synchronously: the Flask /predict route reads 17 form fields, converts age/experience into DAYS_BIRTH/DAYS_EMPLOYED, builds a feature vector, and passes it to the pre-trained Random Forest model, returning a Good/Bad Customer result in the same request-response cycle. No financial transactions are processed, only a risk classification.	High
5	Reporting	The training script (main.py) prints accuracy, F1 score, confusion matrix, and classification report to the console for each model (Logistic Regression, Decision Tree, Random Forest). The live web app itself does not yet generate reports or dashboards for end users.	Medium
6	Business rules, etc.	A customer is labeled 'Bad' if their credit STATUS was ever 1-5 (i.e., 1+ months overdue) in their monthly credit history, and 'Good' (C, X, 0) otherwise, with one final label per customer after grouping by ID. Random Forest was selected as the production model.	High
7	Compliance to laws or regulations	The system processes sensitive personal and financial data (income, family status, employment). It should be brought into alignment with applicable data protection regulations (e.g., India's DPDP Act 2023, or GDPR if used with EU applicants) - this is not yet implemented.	Medium
8	Other	Model training/evaluation (main.py) is kept separate from the serving layer (app.py); the trained model is persisted via pickle and loaded once at app startup for inference.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Since the model is a lightweight pre-trained Random Forest loaded once at startup, a prediction should return almost instantly after form submission.	Prediction response returned in < 2 seconds per request

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
2	Scalability	The app currently runs on Flask's built-in development server (app.run(debug=True)), which is single-threaded and not meant for production load.	Should move to a WSGI server (e.g., Gunicorn) to support at least 100 concurrent users
3	Security & Data Privacy	Form data (income, family status, employment) is currently sent and handled without encryption in transit, and raw exceptions are shown to the user on error.	Serve over HTTPS/TLS 1.2+, sanitize inputs, and avoid exposing raw stack traces to users
4	Reliability & Availability	No uptime guarantee exists in the current local/dev setup; the app has no fault-tolerance or restart mechanism if it crashes.	Target 99% monthly uptime once deployed behind a process manager (e.g., systemd, Docker restart policy)
5	Usability & Accessibility	The form uses plain HTML templates (index.html, result.html) without documented accessibility labeling or validation feedback.	Add clear field labels, inline validation messages, and aim for WCAG 2.1 AA compliance
6	Other	Maintainability: training (main.py) and serving (app.py) are already separated, which should be preserved. Portability: model.pkl is tied to the scikit-learn version it was trained with, so the environment must pin compatible library versions.	Document exact scikit-learn/pandas versions used for training in requirements.txt